AWS Cloud Infrastructure Project

Scalable 3-Tier Web Application on AWS

Job-Ready Portfolio Project • Step-by-Step Guide • Resume-Ready

Services Covered

EC2	IAM	S3	RDS	VPC
Route 53	CloudWatch	Auto Scaling	Lambda	ALB

Prepared: May 2026

Project Overview

You will build a production-grade, scalable 3-tier web application on AWS that demonstrates real-world cloud engineering skills. This is the exact type of project hiring managers and interviewers look for on a resume.

What You Will Build

A fully functional web application with:

- Custom VPC with public and private subnets across 2 Availability Zones
- EC2 instances serving a web application behind an Application Load Balancer
- Auto Scaling Group to automatically scale EC2 instances based on traffic
- RDS MySQL database in a private subnet (Multi-AZ ready)
- S3 bucket for static assets and application storage
- IAM roles and policies following least-privilege security
- Route 53 domain routing to the Load Balancer
- CloudWatch dashboards, alarms, and log groups for full observability
- Lambda function for a serverless backend task (S3 trigger)

Architecture Summary

Project Name	AWS 3-Tier Scalable Web App
Region	us-east-1 (N. Virginia) — Free Tier eligible
Duration	3–5 days (hands-on, step-by-step)
Estimated Cost	~$0 with AWS Free Tier (first 12 months)
Difficulty Level	Beginner to Intermediate
Resume Value	High — covers 10 core AWS services

> 📌 **Note:** Always work in us-east-1 (N. Virginia). It has the most Free Tier coverage and all services are available there.

Prerequisites — Before You Start

Step 1: Create Your AWS Account

If you do not already have an AWS account, follow these steps:

1. **Step 1 —** Go to https://aws.amazon.com and click 'Create an AWS Account'
2. **Step 2 —** Enter your email address, choose a password, and pick an account name
3. **Step 3 —** Select 'Personal' account type and enter your contact information
4. **Step 4 —** Enter a valid credit/debit card (required, but Free Tier means you pay nothing for this project)
5. **Step 5 —** Complete phone verification
6. **Step 6 —** Choose the 'Basic Support' (Free) plan
7. **Step 7 —** Wait for email confirmation (usually 5–10 minutes)

> ☑ **Tip:** Enable MFA (Multi-Factor Authentication) on your root account immediately after signup. Go to IAM → Security Recommendations → Enable MFA.

Step 2: Set Up AWS CLI

AWS CLI lets you manage AWS from your terminal — essential for any cloud engineer.

Install AWS CLI

Windows:

```
# Download from: https://awscli.amazonaws.com/AWSCLIV2.msi
# Run the installer, then open Command Prompt and verify:
aws --version
```

Mac:

```
brew install awscli
aws --version
```

Linux (Ubuntu/Debian):

```
curl 'https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip' -o 'awscliv2.zip'
unzip awscliv2.zip
sudo ./aws/install
aws --version
```

Configure AWS CLI

```
aws configure
# You will be prompted for:
# AWS Access Key ID: [paste your key]
# AWS Secret Access Key: [paste your secret]
# Default region name: us-east-1
# Default output format: json
```

> 📌 **Note:** To get your Access Key: Log in to AWS Console → Click your name (top right) → Security Credentials → Create Access Key.

Phase 1: IAM — Identity and Access Management

IAM controls WHO can access WHAT in your AWS account. Best practice: never use your root account for day-to-day work. Always create dedicated users and roles.

Concepts to Understand

- Users — individual identities (for humans)
- Groups — collection of users with shared permissions
- Roles — identities assumed by AWS services (like EC2)
- Policies — JSON documents defining what is allowed/denied
- Least Privilege — only give the permissions absolutely needed

Step 1.1 — Create an Admin IAM User

Go to AWS Console → IAM → Users → Add Users

- Enter username: aws-admin
- Select 'Provide user access to the AWS Management Console'
- Choose 'Attach policies directly' → check 'AdministratorAccess'
- Click Create User and save the login URL, username, and password

> ☑ **Tip:** From now on, log in using this IAM user, NOT the root account. Protect your root account like a master key.

Step 1.2 — Create an EC2 IAM Role

EC2 instances need a role to access other AWS services (S3, CloudWatch, etc.) without hardcoding credentials.

- Go to IAM → Roles → Create Role
- Trusted Entity Type: AWS Service → EC2
- Attach these policies: AmazonS3ReadOnlyAccess, CloudWatchAgentServerPolicy
- Name the role: EC2-WebApp-Role
- Click Create Role

Step 1.3 — Create a Lambda Execution Role

- IAM → Roles → Create Role
- Trusted Entity: AWS Service → Lambda
- Attach: AWSLambdaBasicExecutionRole + AmazonS3ReadOnlyAccess
- Name: Lambda-S3-Trigger-Role

IAM User Created	aws-admin (AdministratorAccess)

EC2 Role	EC2-WebApp-Role (S3 + CloudWatch)
Lambda Role	Lambda-S3-Trigger-Role
Concept Learned	Least privilege, roles vs users vs groups

Phase 2: VPC — Virtual Private Cloud

A VPC is your own isolated private network in AWS. Everything you build lives inside a VPC. Understanding VPC is one of the most important skills for any AWS role.

Concepts to Understand

- VPC — isolated virtual network (like your own data center network)
- Subnets — subdivisions of a VPC in a specific Availability Zone
- Public Subnet — has a route to the Internet (for EC2 web servers)
- Private Subnet — no direct internet access (for databases)
- Internet Gateway (IGW) — allows internet traffic into/out of your VPC
- Route Table — rules for where traffic should go
- Security Groups — virtual firewall for EC2 instances
- NACL — Network ACL, optional second layer of firewall at subnet level

Step 2.1 — Create the VPC

- Go to VPC Console → Your VPCs → Create VPC
- Name: webapp-vpc
- IPv4 CIDR: 10.0.0.0/16 (gives you 65,536 IP addresses)
- Leave IPv6 as No IPv6 CIDR
- Tenancy: Default
- Click Create VPC

Step 2.2 — Create 4 Subnets

Create subnets across 2 Availability Zones for high availability:

Subnet Name	Type	CIDR	AZ
public-subnet-1a	Public	10.0.1.0/24	us-east-1a
public-subnet-1b	Public	10.0.2.0/24	us-east-1b
private-subnet-1a	Private	10.0.3.0/24	us-east-1a
private-subnet-1b	Private	10.0.4.0/24	us-east-1b

> 📌 **Note:** After creating public subnets, enable 'Auto-assign public IPv4' on them: select subnet → Actions → Edit subnet settings → Enable Auto-assign public IPv4.

Step 2.3 — Create and Attach Internet Gateway

- VPC → Internet Gateways → Create Internet Gateway

- Name: webapp-igw
- After creation: Actions → Attach to VPC → select webapp-vpc

Step 2.4 — Configure Route Tables

Public Route Table (for internet access):

- VPC → Route Tables → Create Route Table
- Name: public-rt, VPC: webapp-vpc
- Select public-rt → Routes → Edit Routes → Add route: 0.0.0.0/0 → Target: webapp-igw
- Subnet Associations → Edit → select both public subnets

Private Route Table (no internet):

- Create another route table: private-rt
- No additional routes needed (private = internal only)
- Associate both private subnets with private-rt

Step 2.5 — Create Security Groups

ALB Security Group (alb-sg):

- Allow Inbound: HTTP (80) from 0.0.0.0/0
- Allow Inbound: HTTPS (443) from 0.0.0.0/0

EC2 Security Group (ec2-sg):

- Allow Inbound: HTTP (80) from alb-sg only (not the whole internet)
- Allow Inbound: SSH (22) from your IP only (for management)

RDS Security Group (rds-sg):

- Allow Inbound: MySQL/Aurora (3306) from ec2-sg only

> ☑ **Tip:** Restricting EC2 to only accept traffic from ALB is a key security best practice — interviewers love this.

Phase 3: EC2 — Elastic Compute Cloud

EC2 is AWS's virtual machine service. You will launch a web server that serves a simple web page.

Step 3.1 — Create a Key Pair

- EC2 Console → Key Pairs → Create Key Pair
- Name: webapp-key
- Type: RSA, Format: .pem (Mac/Linux) or .ppk (Windows with PuTTY)
- Download and save securely — you cannot download it again

Step 3.2 — Launch EC2 Instance

- EC2 → Instances → Launch Instances
- Name: WebServer-1
- AMI: Amazon Linux 2023 (Free Tier eligible)
- Instance Type: t2.micro (Free Tier)
- Key Pair: webapp-key
- Network: webapp-vpc, Subnet: public-subnet-1a
- Security Group: ec2-sg
- IAM Instance Profile: EC2-WebApp-Role

Step 3.3 — User Data Script (Auto-install web server)

In the Advanced Details section → User Data, paste this script. It runs automatically when the instance starts:

```
#!/bin/bash
yum update -y
yum install -y httpd
systemctl start httpd
systemctl enable httpd
EC2_AZ=$(curl -s http://169.254.169.254/latest/meta-data/placement/availability-zone)
echo '<html><body style="font-family:Arial;text-align:center;padding:50px;">' > /var/www/html/index.html
echo '<h1 style="color:#1F4E79;">AWS 3-Tier Web App</h1>' >> /var/www/html/index.html
echo "<p>Running in: <b>$EC2_AZ</b></p>" >> /var/www/html/index.html
echo '<p>EC2 + ALB + Auto Scaling + RDS + S3</p>' >> /var/www/html/index.html
echo '</body></html>' >> /var/www/html/index.html
```

> ☑ **Tip:** After launch, wait 2 minutes, then copy the Public IP of your instance and paste it in a browser. You should see your web page!

Step 3.4 — Connect via SSH

To log in to your EC2 instance from terminal:

```
# Mac / Linux:
chmod 400 webapp-key.pem
ssh -i webapp-key.pem ec2-user@<YOUR-EC2-PUBLIC-IP>

# Windows (PowerShell):
ssh -i webapp-key.pem ec2-user@<YOUR-EC2-PUBLIC-IP>
```

Instance Name	WebServer-1
AMI	Amazon Linux 2023
Type	t2.micro (Free Tier)
Subnet	public-subnet-1a
IAM Role	EC2-WebApp-Role
Web Server	Apache HTTPD (auto-started via User Data)

Phase 4: S3 — Simple Storage Service

S3 is AWS object storage — used for images, files, backups, static websites, and more. It is infinitely scalable and highly durable (99.999999999% durability).

Step 4.1 — Create an S3 Bucket

- Go to S3 Console → Create Bucket
- Bucket name: webapp-assets-[yourname]-2026 (must be globally unique)
- Region: us-east-1
- Block all public access: KEEP ENABLED (default is secure)
- Enable Versioning: ON (for data protection)
- Click Create Bucket

Step 4.2 — Upload a Test File

- Click into your bucket → Upload → Add Files
- Upload any small image or text file
- Click Upload

Step 4.3 — Access S3 from EC2 Using IAM Role

Because you attached the EC2-WebApp-Role with S3 access, you can access S3 from your EC2 instance without any credentials:

```
# SSH into your EC2 instance first, then:
aws s3 ls s3://webapp-assets-[yourname]-2026

# Copy a file from S3 to EC2:
aws s3 cp s3://webapp-assets-[yourname]-2026/yourfile.txt /tmp/
```

> ☑ **Tip:** This is the correct, secure way to give EC2 access to S3 — via IAM Role, never by storing AWS keys on the instance.

Step 4.4 — Enable S3 Static Website Hosting (Optional)

- Bucket → Properties → Static website hosting → Enable
- Index document: index.html
- Upload an index.html file
- Bucket Policy → Edit → paste allow-public-read policy

```
{"Version":"2012-10-17","Statement":[{"Effect":"Allow",
"Principal":"*","Action":"s3:GetObject",
"Resource":"arn:aws:s3:::webapp-assets-[yourname]-2026/*"}]}
```

Phase 5: RDS — Relational Database Service

RDS manages your database server. AWS handles backups, patching, and failover. You just focus on your data.

Step 5.1 — Create a DB Subnet Group

RDS needs to know which subnets to use (always use private subnets for databases):

- RDS Console → Subnet Groups → Create DB Subnet Group
- Name: webapp-db-subnet-group
- VPC: webapp-vpc
- Add Subnets: select private-subnet-1a and private-subnet-1b
- Click Create

Step 5.2 — Create the RDS Database

- RDS → Databases → Create Database
- Engine: MySQL 8.0
- Template: Free Tier
- DB Instance ID: webapp-db
- Master Username: admin
- Master Password: [choose a strong password, save it!]
- Instance class: db.t3.micro (Free Tier)
- Storage: 20 GB gp2
- VPC: webapp-vpc, Subnet Group: webapp-db-subnet-group
- Public Access: NO (private subnet only)
- Security Group: rds-sg
- Initial DB name: webappdb
- Click Create Database (takes 5–10 minutes)

> 📌 **Note:** The RDS endpoint will look like: webapp-db.xxxxxxxx.us-east-1.rds.amazonaws.com — copy this, your app needs it to connect.

Step 5.3 — Test Database Connection from EC2

```
# SSH into your EC2 instance
sudo yum install -y mysql

# Connect to RDS (replace endpoint with your RDS endpoint):
mysql -h webapp-db.xxxxxxxx.us-east-1.rds.amazonaws.com -u admin -p

# After connecting, create a test table:
USE webappdb;
CREATE TABLE visitors (id INT AUTO_INCREMENT PRIMARY KEY, visit_time DATETIME);
INSERT INTO visitors (visit_time) VALUES (NOW());
```

```
SELECT * FROM visitors;
```

> ☑ **Tip:** RDS in a private subnet + EC2 in public subnet is the standard 3-tier pattern. This shows interviewers you understand secure architecture.

Phase 6: Application Load Balancer + Auto Scaling

This phase makes your application highly available and scalable — it automatically adds or removes EC2 instances based on traffic load.

Step 6.1 — Create a Launch Template

A Launch Template defines what each new EC2 instance should look like:

- EC2 → Launch Templates → Create Launch Template
- Name: webapp-launch-template
- AMI: Amazon Linux 2023, Instance type: t2.micro
- Key pair: webapp-key, Security group: ec2-sg
- IAM Instance Profile: EC2-WebApp-Role
- User Data: paste the same script from Phase 3

Step 6.2 — Create Application Load Balancer

- EC2 → Load Balancers → Create Load Balancer → Application Load Balancer
- Name: webapp-alb
- Scheme: Internet-facing
- VPC: webapp-vpc, Subnets: public-subnet-1a and public-subnet-1b
- Security Group: alb-sg
- Listener: HTTP:80
- Create a new Target Group: webapp-tg, Target type: Instances, Protocol: HTTP:80
- Click Create Load Balancer

Step 6.3 — Create Auto Scaling Group

- EC2 → Auto Scaling Groups → Create
- Name: webapp-asg
- Launch Template: webapp-launch-template
- VPC: webapp-vpc, Subnets: public-subnet-1a, public-subnet-1b
- Attach to existing load balancer: webapp-tg
- Min capacity: 1, Desired: 2, Max: 4
- Scaling Policy: Target tracking → Average CPU utilization: 50%
- Click Create Auto Scaling Group

> ☑ **Tip:** Copy the DNS name of your ALB (e.g. webapp-alb-123456.us-east-1.elb.amazonaws.com) and open it in a browser. Refresh multiple times — you should see different Availability Zones in your web page!

ALB Name	webapp-alb (internet-facing)
Target Group	webapp-tg (port 80)

ASG Min/Desired/Max	1 / 2 / 4
Scaling Policy	CPU > 50% = scale out
Result	Zero-downtime, auto-scaling web app

Phase 7: Route 53 — DNS Management

Route 53 is AWS's DNS service. It routes internet users to your application. Even without a custom domain, you can explore Route 53 with the ALB DNS name.

Option A: Use a Custom Domain (Recommended)

If you have a domain (or buy one from Route 53 for ~$12/year):

- Route 53 → Hosted Zones → Create Hosted Zone
- Domain: yourdomain.com, Type: Public
- Create a Record → A record (Alias)
- Route traffic to: Application Load Balancer → us-east-1 → select webapp-alb
- Record name: www (or leave blank for root domain)
- Click Create Records

Option B: No Domain — Test Health Checks

You can still explore Route 53 without buying a domain:

- Route 53 → Health Checks → Create Health Check
- Name: webapp-health-check
- Protocol: HTTP, Endpoint: IP address of your ALB
- Port: 80, Path: /
- Click Create Health Check
- Watch it turn green (Healthy) after 1–2 minutes

> 📌 **Note:** For the resume, you can describe Route 53 as 'Configured Route 53 DNS routing and health checks for ALB endpoint' — this shows knowledge even without owning a domain.

Phase 8: CloudWatch — Monitoring & Observability

CloudWatch is your eyes into the AWS infrastructure. Real-world cloud engineers spend a lot of time in CloudWatch. This phase will make you comfortable with metrics, alarms, and logs.

Step 8.1 — Create a CloudWatch Dashboard

- CloudWatch → Dashboards → Create Dashboard
- Name: WebApp-Dashboard
- Add widgets:
- - EC2 CPUUtilization (Line graph) for your ASG instances
- - EC2 NetworkIn and NetworkOut
- - RDS CPUUtilization and DatabaseConnections
- - ALB RequestCount and TargetResponseTime
- Click Save Dashboard

Step 8.2 — Create CloudWatch Alarms

High CPU Alarm:

- CloudWatch → Alarms → Create Alarm
- Metric: EC2 → Per-Instance Metrics → CPUUtilization
- Condition: Greater than 80% for 2 consecutive periods
- Action: Create new SNS topic → enter your email → subscribe to get alerts
- Alarm name: HighCPU-EC2

RDS Storage Alarm:

- Metric: RDS → Per-Database Metrics → FreeStorageSpace
- Condition: Less than 5GB (5000000000 bytes)
- Alarm name: LowStorage-RDS

Step 8.3 — Enable CloudWatch Logs for EC2

Install and configure the CloudWatch Agent on your EC2 instance:

```
# SSH into EC2 and install CloudWatch agent:
sudo yum install -y amazon-cloudwatch-agent

# Create config file:
sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-config-wizard

# Start the agent:
sudo systemctl start amazon-cloudwatch-agent
sudo systemctl enable amazon-cloudwatch-agent
```

> ☑ **Tip:** After setup, go to CloudWatch → Log Groups → you will see /var/log/httpd/access_log and error logs streaming in real time.

Phase 9: Lambda — Serverless Computing

Lambda lets you run code without managing any servers. You only pay per execution (100ms increments). You will create a Lambda function that triggers whenever a file is uploaded to S3.

Step 9.1 — Create the Lambda Function

- Lambda Console → Create Function
- Function name: S3-Upload-Notifier
- Runtime: Python 3.12
- Execution role: Use existing role → Lambda-S3-Trigger-Role
- Click Create Function

Step 9.2 — Write the Function Code

In the Code Editor, replace the default code with:

```
import json
import boto3
from datetime import datetime

def lambda_handler(event, context):
    print('Lambda triggered at:', datetime.now().isoformat())

    for record in event.get('Records', []):
        bucket = record['s3']['bucket']['name']
        key = record['s3']['object']['key']
        size = record['s3']['object'].get('size', 0)

        print(f'New file uploaded!')
        print(f'Bucket: {bucket}')
        print(f'File: {key}')
        print(f'Size: {size} bytes')

    return {
        'statusCode': 200,
        'body': json.dumps('S3 event processed successfully!')
    }
```

Click Deploy to save.

Step 9.3 — Add S3 Trigger

- Lambda function page → Add Trigger
- Source: S3
- Bucket: webapp-assets-[yourname]-2026
- Event type: All object create events
- Click Add

Step 9.4 — Test It

- Go to S3 → upload any file to your bucket
- Go to Lambda → Monitor → View CloudWatch Logs
- You will see your print statements with the file name and size

> ☑ **Tip:** Lambda + S3 trigger is a very common real-world pattern used for image processing, data pipelines, and notifications.

Resume & Interview — How to Present This Project

Resume Bullet Points

Add these to your resume under 'Projects':

AWS 3-Tier Scalable Web Application

- Designed and deployed a 3-tier web application on AWS using EC2, ALB, Auto Scaling, RDS (MySQL), S3, Route 53, CloudWatch, and Lambda
- Built a custom VPC with public/private subnets across 2 Availability Zones for high availability and security isolation
- Configured IAM roles with least-privilege policies; eliminated credential hardcoding using EC2 instance profiles
- Implemented Auto Scaling Group with target tracking policy (CPU > 50%) enabling automatic horizontal scaling
- Set up CloudWatch dashboards, metric alarms (CPU, storage), and log streaming via CloudWatch Agent on EC2
- Created serverless S3 event processing pipeline using Lambda (Python) triggered on object uploads

Common Interview Questions & Answers

Q: What is the difference between a Security Group and a NACL?

Security Groups are stateful (return traffic is automatically allowed) and operate at the instance level. NACLs are stateless (you must explicitly allow return traffic) and operate at the subnet level. In this project, we used Security Groups to restrict EC2 to only accept HTTP from the ALB.

Q: Why did you put RDS in a private subnet?

Databases should never be directly accessible from the internet. By placing RDS in a private subnet with a security group that only allows traffic from the EC2 security group on port 3306, we follow the principle of defense in depth.

Q: How does Auto Scaling work?

The Auto Scaling Group monitors CloudWatch metrics (CPU utilization). When CPU exceeds the threshold (50%), it launches new EC2 instances from the Launch Template into available subnets. When load drops, it terminates excess instances. The ALB automatically registers/deregisters instances as they launch or terminate.

Q: What is the difference between S3 and EBS?

EBS (Elastic Block Store) is a block storage device attached to one EC2 instance — like a hard drive. S3 is object storage accessible from anywhere, by multiple services simultaneously. S3 is used for files, images, backups, and static hosting. EBS is used for the root volume of EC2 or database storage.

Q: Why use Lambda instead of EC2 for the S3 trigger?

Lambda is ideal for event-driven, short-duration tasks. It scales to zero (no cost when idle), scales automatically with events, and requires no server management. An EC2 instance running 24/7 to poll for S3 events would be wasteful and more expensive.

Cleanup — Avoid Unexpected Charges

When you are done with the project, delete resources in this order to avoid charges:

1. Auto Scaling Group	Set min/desired/max to 0, then delete
2. Application Load Balancer	Delete webapp-alb
3. Target Group	Delete webapp-tg
4. EC2 Instances	Terminate any running instances
5. RDS Database	Delete webapp-db (uncheck final snapshot for lab)
6. RDS Subnet Group	Delete webapp-db-subnet-group
7. S3 Bucket	Empty bucket first, then delete
8. Lambda Function	Delete S3-Upload-Notifier
9. CloudWatch	Delete dashboards and alarms
10. VPC	Delete subnets, route tables, IGW, security groups, then VPC
11. IAM Roles	Delete EC2-WebApp-Role, Lambda-S3-Trigger-Role

> 📌 **Note:** Go to AWS Cost Explorer and Billing Dashboard after cleanup to confirm $0 charges. Set a billing alarm in CloudWatch to alert you if charges exceed $5/month.

Skills Summary — What You Have Learned

Service	What You Did	Concept Mastered
IAM	Users, roles, policies	Least privilege, secure access
VPC	Custom network, subnets, routing	Network isolation, AZ design
EC2	Web server, user data, SSH	Virtual machines, instance profiles
S3	Bucket, versioning, CLI access	Object storage, IAM-based access
RDS	MySQL in private subnet	Managed databases, 3-tier pattern
ALB	Load balancer, target group	Traffic distribution, health checks
Auto Scaling	Launch template, ASG, policies	Horizontal scaling, HA design
Route 53	DNS, health checks	Domain routing, failover
CloudWatch	Dashboard, alarms, logs	Observability, alerting
Lambda	Python function, S3 trigger	Serverless, event-driven compute

Congratulations! You have built a job-ready AWS cloud project. Add it to your resume, push screenshots to GitHub, and you are ready to interview.